# Cybersecurity Command Cheatsheet

*A small, safe reference for learning paths: Windows, Linux, logs, networking, and local web labs*

**NOTEBOOK STYLE** Handwriting-inspired typeset notes for printing and revision; not a scan of actual handwriting.

**SAFE USE** Use these commands on your own device, supplied sample files, and a local training app. Do not test public systems without explicit permission.

## 1. Windows basics (PowerShell)

| Goal | Read-only command / example |
|---|---|
| See running processes | Get-Process \| Sort-Object CPU -Descending \| Select-Object -First 10 Name,Id,CPU |
| Read recent authentication events (permissions may be required) | Get-WinEvent -FilterHashtable @{LogName="Security"; Id=4624,4625} -MaxEvents 20 |
| Read JSON training data | $data = Get-Content .\sample.json -Raw \| ConvertFrom-Json |
| Find a word in a sample file | Select-String -Path .\sample.csv -Pattern "suspicious" |

## 2. Linux basics

| Goal | Command / example |
|---|---|
| Show addresses on your own device | ip addr |
| List listening sockets on your own device | ss -tulpn |
| Read recent warning-level journal entries | journalctl -p warning -n 50 |
| Search a local log file for a word | grep -in "failed" sample.log |
| Count repeated first-column values (CSV must be simple) | cut -d, -f1 sample.csv \| sort \| uniq -c |

## 3. Networking concepts

| Question | Safe example |
|---|---|
| What address does a public training domain resolve to? | nslookup example.com |
| What HTTP headers does my local training app return? | curl -I http://127.0.0.1:3000 |
| What is a port? | A numbered endpoint used by a network service; do not assume an open port is a vulnerability. |
| What is DNS? | A naming system that helps map names to records such as IP addresses. |

## 4. Log triage workflow

1. Record the time range and timezone.
2. Filter the provided synthetic sample to the event types relevant to the question.
3. Group by user, host, source, and outcome where useful.
4. Compare successful and failed activity with a baseline.
5. Keep raw events and note which filters you applied.
6. Write facts, hypotheses, unknowns, and next steps separately.

## 5. Threat-hunting memory aid

**HYPOTHESIS → QUERY → VALIDATE → DOCUMENT** What would I expect if my hypothesis were true? What else could explain the same pattern? What data is missing?

## 6. Bug-bounty checklist

- Read the program brief, exact in-scope assets, exclusions, rate limits, and disclosure rules.
- Prefer official training platforms or a local lab for practice.
- Use minimal, non-destructive tests and demo accounts only.
- Stop if real user data or a production-side effect appears.
- Write a reproducible report and redact secrets, tokens, and personal data.

## 7. Useful formulas

| Metric | Formula | Interpretation |
|---|---|---|
| Precision | TP / (TP + FP) | Of the rule alerts, how many were true positives in labelled data? |
| Recall | TP / (TP + FN) | How many labelled suspicious cases did it find? |
| Specificity | TN / (TN + FP) | How many labelled benign cases did it ignore correctly? |
| F1 | 2 × Precision × Recall / (Precision + Recall) | A combined measure of precision and recall. |